

Training a Flappy Bird Agent with Deep Q-Networks in PyTorch

Tran Van Thuan
FPT University
tranvanthuanppr@gmail.com

Nguyen Vo Hoang Khang
FPT University
khangnhse184634@fpt.edu.vn

Abstract—This paper presents the implementation and training of a reinforcement learning agent for playing Flappy Bird using the Deep Q-Network (DQN) algorithm. The system is implemented in PyTorch using the *flappy_bird_gymnasium* environment and includes experience replay, a target network, epsilon-greedy action selection, and optional Double DQN and Dueling DQN extensions. Four training configurations are evaluated based on their best observed reward, sample efficiency, and wall-clock training time: a baseline run (*flappybird1*), a Double DQN run (*flappybird2*), a Dueling DQN run (*flappybird3*), and a combined Double+Dueling DQN run (*flappybird4*). Experimental results show that *flappybird2* (Double DQN only) achieves the highest best reward of 278.9 after 403,527 episodes and the highest reward-per-hour rate (53.2), narrowly ahead of *flappybird3* (272.8 after 548,802 episodes). Combining Double and Dueling DQN in *flappybird4* does not further improve performance, reaching only 204.4. All three tuned configurations substantially outperform the initial baseline configuration, which reaches a best reward of 106.4 after approximately 941,863 episodes and 23.09 hours of training.

Index Terms—Deep Q-Network, Reinforcement Learning, Flappy Bird, PyTorch, Double DQN, Dueling DQN, Experience Replay.

I. INTRODUCTION

Reinforcement learning (RL) is an important approach in artificial intelligence in which an agent learns to make decisions through interaction with an environment. At each time step, the agent observes the current state, selects an action, receives a reward, and updates its behavior in order to maximize the expected cumulative reward.

Flappy Bird is a suitable environment for RL experiments because its rules are simple while the control task remains challenging. The agent must continuously decide when to flap in order to avoid pipes and survive for as long as possible. This creates a compact but meaningful benchmark for testing value-based reinforcement learning methods.

The objective of this work is to implement DQN from scratch in PyTorch, without relying on high-level RL libraries such as Stable-Baselines3, and to examine how design choices such as Double DQN, Dueling DQN, replay buffer size, mini-batch size, learning rate, and loss function affect training performance.

II. BACKGROUND

A. Q-Learning

Q-learning estimates the action-value function $Q(s, a)$, which represents the expected return obtained by taking action

a in state s . The Bellman target is defined as:

$$y = r + \gamma \max_{a'} Q(s', a'), \quad (1)$$

where r is the immediate reward, s' is the next state, and γ is the discount factor.

B. Deep Q-Network

DQN approximates the action-value function using a neural network. Instead of storing a Q-table for individual states, the network receives the state vector as input and outputs a Q-value for each available action. To stabilize training, DQN commonly uses experience replay and a target network. Experience replay stores transitions $(s, a, r, s', done)$, while the target network is periodically synchronized from the policy network to reduce instability in the learning target.

C. Double DQN

Standard DQN can overestimate Q-values because the same network is used both to select and evaluate the next action. Double DQN separates these two steps: the policy network selects the best next action, while the target network evaluates that action:

$$y = r + \gamma Q_{\text{target}}(s', \arg \max_{a'} Q_{\text{policy}}(s', a')). \quad (2)$$

D. Dueling DQN

Dueling DQN decomposes the network into two streams: a state-value stream $V(s)$ and an advantage stream $A(s, a)$. The Q-value is computed as:

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'). \quad (3)$$

This architecture helps the model learn the value of a state even when the advantage differences among actions are small.

III. IMPLEMENTATION METHOD

A. Environment and Action Space

The environment used in this project is *FlappyBird-v0* from the *flappy_bird_gymnasium* library. With *use_lidar=False*, the observation is a 12-dimensional state vector describing the bird and pipe-related variables. The action space contains two discrete actions:

- 0: do nothing;
- 1: flap.

B. Network Architecture

The DQN model is implemented in *dqn.py*. In the main configurations, the network receives a 12-dimensional state vector and passes it through a fully connected layer with 512 hidden units followed by a ReLU activation. When Dueling DQN is enabled, the hidden representation is split into a value stream and an advantage stream. When Dueling DQN is disabled, a single linear output layer directly produces the Q-values for the two actions.

C. Training Loop

The agent is implemented in *agent.py*. At each episode, the agent resets the environment, selects actions using an epsilon-greedy policy, stores transitions in replay memory, and optimizes the policy network once enough samples are available. Epsilon decays multiplicatively until it reaches a predefined minimum value, allowing the agent to gradually shift from exploration to exploitation.

The current implementation uses Smooth L1 Loss, also known as Huber loss:

$$\mathcal{L} = \text{SmoothL1}(Q_{\text{policy}}(s, a), y). \quad (4)$$

The network is optimized using Adam. The target network is synchronized from the policy network after a fixed number of environment steps controlled by the *network_sync_rate* parameter.

IV. EXPERIMENTAL SETUP

Table I summarizes the four configurations compared in this work. The *flappybird1* configuration corresponds to the original baseline run that produced the recorded checkpoint and log; note that the *hyperparameters.yml* file on disk was later edited, so the Double DQN and loss values shown for *flappybird1* reflect the settings actually used to produce that run rather than the current file contents. *flappybird2* adds Double DQN with the tuned buffer/batch/learning-rate settings, *flappybird3* replaces Double DQN with Dueling DQN under the same tuned settings, and *flappybird4* enables both extensions together.

V. RESULTS

The training logs show that all four configurations improve over time, but they differ substantially in best reward, sample efficiency, and wall-clock training time. Table II summarizes the recorded results, including reward achieved per hour of training as a measure of overall efficiency.

Fig. ??, Fig. ??, and Fig. ?? show the training curves for the individual runs and a comparison across all four runs. In *flappybird1*, the best reward increases slowly and reaches 106.4 at episode 941,863. In *flappybird3*, the reward improves more rapidly, reaching 116.1 at episode 174,288 and 272.8 at episode 548,802. Among the four configurations, *flappybird2* (Double DQN only) attains both the highest best reward (278.9) and, by a wide margin, the highest reward-per-hour rate (53.2), reaching that reward in roughly a quarter of the wall-clock time required by the baseline. *flappybird3* (Dueling

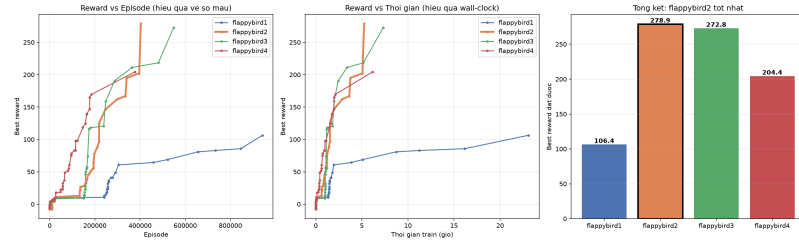


Fig. 1. Compare 4 version

DQN only) is a close second in best reward. Notably, *flappybird4*, which combines Double and Dueling DQN, does not outperform either single-extension configuration, suggesting that the two extensions do not compound additively in this environment.

VI. DISCUSSION

The gap between *flappybird1* and the three tuned configurations (*flappybird2*–*flappybird4*) indicates that hyperparameter tuning has a significant effect on learning performance. A larger replay memory allows the agent to sample from a more diverse set of experiences. A larger mini-batch size can provide more stable gradient estimates, while a lower learning rate may reduce unstable updates. The higher discount factor used in the tuned runs also encourages the agent to focus more on long-term survival, which matches the objective of Flappy Bird.

Comparing *flappybird2*, *flappybird3*, and *flappybird4*, which share identical replay buffer, batch size, learning rate, and discount factor but differ only in the Double/Dueling DQN switches, isolates the effect of the two extensions. Both Double DQN alone (*flappybird2*) and Dueling DQN alone (*flappybird3*) perform similarly well and clearly outperform the baseline, with Double DQN reaching a slightly higher best reward at much lower wall-clock cost. However, enabling both extensions simultaneously (*flappybird4*) yields the lowest best reward among the three tuned runs. This suggests that, in this environment and with this network size, the two extensions do not combine additively and may instead interact in ways that slow convergence, such as increased variance from correcting overestimation (Double DQN) while simultaneously reparameterizing the output head (Dueling DQN).

The results also suggest that Dueling DQN is useful in an environment with few actions but meaningful state values. In many states, whether the bird is in a safe or dangerous position may be more important than the immediate difference between the two available actions. The dueling architecture gives the model a more direct way to represent this property, though the experiments here do not show a clear advantage of Dueling DQN over Double DQN in isolation.

One limitation of the experiment is that random seeds were not fixed across the full training pipeline and each configuration was trained only once, so results may vary between independent runs. In addition, the current analysis is mainly based on the best reward and reward curves. A more rigorous evaluation would include multiple runs per

TABLE I
MAIN HYPERPARAMETER CONFIGURATIONS

Parameter	flappybird1	flappybird2	flappybird3	flappybird4
Replay memory	100,000	200,000	200,000	200,000
Mini-batch size	32	64	64	64
ϵ_{init}	1.00	1.00	1.00	1.00
ϵ_{min}	0.05	0.01	0.01	0.01
Epsilon decay	0.99995	0.99995	0.99995	0.99995
Sync rate	10	100	100	100
Learning rate	0.0001	0.00005	0.00005	0.00005
Discount factor γ	0.99	0.995	0.995	0.995
Hidden nodes	512	512	512	512
Double DQN	Yes	Yes	No	Yes
Dueling DQN	No	No	Yes	Yes
Loss	MSE (original record)	Smooth L1	Smooth L1	Smooth L1

TABLE II
RECORDED TRAINING RESULTS

Run	Double	Dueling	Best reward	Episodes	Time (h)	Reward/hour
flappybird1	Yes	No	106.4	941,863	23.09	4.6
flappybird2	Yes	No	278.9	403,527	5.25	53.2
flappybird3	No	Yes	272.8	548,802	7.33	37.2
flappybird4	Yes	Yes	204.4	377,378	6.17	33.1

configuration, mean performance, standard deviation, and test-time evaluation using saved checkpoints.

VII. CONCLUSION

This paper presented a PyTorch implementation of DQN for training an agent to play Flappy Bird. The system uses core DQN components such as experience replay, a target network, epsilon-greedy exploration, and optional Double DQN and Dueling DQN extensions. The experimental results across four configurations show that both Double DQN (*flappybird2*, best reward 278.9) and Dueling DQN (*flappybird3*, best reward 272.8) individually and substantially outperform the untuned baseline (*flappybird1*, best reward 106.4), with Double DQN reaching its result in a fraction of the training time. Combining both extensions (*flappybird4*, best reward 204.4) did not yield further improvement. Future work may include fixed-seed experiments, repeated evaluations per configuration, Prioritized Experience Replay, and comparisons with policy-gradient methods such as PPO or A2C.

REFERENCES

- [1] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [2] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in *Proc. AAAI*, 2016, pp. 2094–2100.
- [3] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, “Dueling network architectures for deep reinforcement learning,” in *Proc. ICML*, 2016, pp. 1995–2003.
- [4] PyTorch Contributors, “PyTorch documentation,” [Online]. Available: <https://pytorch.org/docs/stable/index.html>.
- [5] Farama Foundation, “Gymnasium documentation,” [Online]. Available: <https://gymnasium.farama.org/>.